

**«Санкт-Петербургский государственный электротехнический
университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы

Студент гр. 4382

А.Д. Смирнов

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: А.Д. Смирнов

Группа: 4382

Тема практики: Генетические алгоритмы

Задание на практику:

Дано N точек в двумерном евклидовом пространстве. Необходимо найти координаты левого нижнего угла и длины сторон для M (задается пользователем) квадратов такие, чтобы квадраты суммарно содержали максимальное количество точек внутри и не пересекались между собой.

Программа должна иметь GUI, через который можно задать чтение данных из файла или их случайную генерацию. Алгоритмы реализуются самостоятельно без использования готовых решений. Настройка параметров алгоритмов должна производиться пользователем. Должна присутствовать пошаговая визуализация поиска решения с переключением между разными текущими решениями и возможностью перехода к конечному решению, пропуская отображение всех шагов. Должен присутствовать график изменения функции качества решения от номера популяции, который дополняется с каждым шагом алгоритма.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент

А.Д. Смирнов

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В данном проекте реализован генетический алгоритм для решения задачи покрытия точек непересекающимися квадратами. Программы написаны на языке *Python* с использованием библиотек *PyQt5* для создания графического интерфейса и *Matplotlib* для отрисовки графиков. Проект разрабатывался с использованием системы контроля версий *git* и платформы *GitHub*.

SUMMARY

This project implements a genetic algorithm to solve the problem of covering points with disjoint squares. The programs are written in Python using the PyQt5 library to create a GUI and the Matplotlib library to draw graphs. The project was developed using the git version control system and the GitHub platform.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
1.3 Третий раздел первой главы	6
1.3.1 Первый подраздел	6
2 РЕЗУЛЬТАТЫ РАБОТЫ	7
2.1. Формулировка задачи 1	7
2.2. Формулировка задачи 2	7
2.3. Формулировка задачи 3	7
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	8
4 ОТЗЫВ РУКОВОДИТЕЛЯ	9
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11
ПРИЛОЖЕНИЕ А	12

ВВЕДЕНИЕ

Предметной областью учебной практики являются генетические алгоритмы и их применение для решения задач оптимизации. Генетические алгоритмы относятся к эвристическим методам поиска решений и используются в случаях, когда полный перебор вариантов является слишком трудоемким или практически невозможным. В рамках данной работы генетический алгоритм применяется для решения задачи покрытия множества точек на плоскости набором непересекающихся квадратов.

Целью практики является разработка программы с графическим интерфейсом для поиска приближенного решения задачи покрытия точек непересекающимися квадратами. Программа должна позволять задавать входные данные, настраивать параметры генетического алгоритма, визуализировать текущие решения и отображать изменение функции качества по поколениям.

Для достижения цели были поставлены следующие задачи: изучить основные принципы работы генетических алгоритмов, разработать структуру графического интерфейса, реализовать ввод исходных точек, подготовить визуализацию точек и квадратов, заложить класс для хранения параметров генетического алгоритма и подготовить проект к дальнейшей реализации операторов отбора, скрещивания, мутации и вычисления функции качества.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы применяются для решения оптимизационных задач, в которых требуется найти достаточно хорошее решение среди большого числа возможных вариантов. Такие методы используются при планировании, размещении объектов, подборе параметров, маршрутизации и других задачах, где невозможно эффективно проверить все возможные комбинации.

В данной работе рассматривается задача покрытия точек квадратами. На вход подаётся набор точек на плоскости и количество квадратов, которое задаётся пользователем. Требуется подобрать координаты левых нижних углов квадратов и длины их сторон так, чтобы суммарно покрыть максимальное количество точек. При этом квадраты не должны пересекаться между собой.

Задача является оптимизационной, так как необходимо максимизировать значение функции качества. В качестве функции качества используется количество покрытых точек с учётом штрафа за пересечение квадратов.

1.2 Задачи практики

В рамках учебной практики необходимо выполнить следующие задачи:

1. Сформировать структуру проекта и выбрать средства разработки.
2. Разработать прототип графического интерфейса.
3. Реализовать ввод исходных данных.
4. Реализовать визуализацию точек и квадратов.
5. Добавить настройку параметров генетического алгоритма через GUI.
6. Подготовить структуру класса генетического алгоритма.
7. Реализовать генетический алгоритм.

8. Реализовать пошаговую визуализацию работы алгоритма.
9. Реализовать график изменения функции качества.

Алгоритмическая часть представлена классом *GeneticAlgorithm*, который управляет процессом поиска. Параметры алгоритма вынесены в класс *GAParams*. Возможное решение задачи описывается классом *Individual*, который содержит набор объектов *Square*. Несколько решений одного поколения объединяются в класс *Population*.

Для вычислительной части выделены классы *Geometry* и *FitnessFunction*. Класс *Geometry* выполняет геометрические проверки: попадание точки в квадрат, пересечение квадратов и подсчёт покрытых точек. Класс *FitnessFunction* вычисляет значение функции качества решения.

Генетические операторы вынесены в отдельные классы. За отбор родителей отвечает интерфейс *SelectionStrategy* и его реализации *TournamentSelection*, *RouletteSelection* и *RankSelection*. Скрещивание выполняется классом *Crossover*, мутация — классом *Mutation*. Для хранения результатов по поколениям используется список популяций внутри класса *GeneticAlgorithm*. Каждая популяция содержит номер поколения, список особей, лучшую особь, лучшее значение функции качества и среднее значение функции качества.

2.2. Разработка прототипа графического интерфейса

Был разработан прототип графического интерфейса приложения с использованием библиотеки *PyQt5*. Главное окно реализовано в классе *MainWindow*. Интерфейс был разделён на три основные области. В левой части окна расположены элементы для ввода исходных данных и настройки параметров алгоритма. В центральной части находится область визуализации, предназначенная для отображения точек и квадратов. В правой части размещён график функции качества, который будет использоваться при пошаговой работе генетического алгоритма. В интерфейсе были подготовлены кнопки для генерации точек, загрузки данных, ручного ввода точек, запуска алгоритма, перехода по шагам и выбора текущего индивида. Часть кнопок на текущем этапе уже получила обработчики, а часть оставлена для дальнейшей реализации.

2.3. Реализация ввода исходных точек

На текущей итерации были реализованы два способа задания исходных точек: случайная генерация и ручной ввод. Для хранения точек, введенных пользователем до запуска алгоритма, в классе *MainWindow* используется поле *input_points*. Такой подход позволяет сначала подготовить входные данные в интерфейсе, отобразить их на сцене, а затем передать в объект класса *GeneticAlgorithm* при запуске алгоритма. Случайная генерация точек вынесена во вспомогательный класс *DataUtils*. Это позволяет отделить логику создания данных от логики графического интерфейса. После генерации точки сохраняются в *input_points*, обновляется информация о количестве точек и вызывается перерисовка области визуализации. Также был реализован ручной ввод точек. Пользователь может ввести координаты самостоятельно, после чего программа преобразует введенный текст в список точек и отображает их в окне визуализации. Это удобно для проверки работы программы на небольших тестовых примерах.

2.4. Реализация визуализации

Для отображения геометрических объектов был реализован класс *VisualWidget*. Он отвечает за отрисовку точек и квадратов в центральной части окна. Метод *set_data()* принимает список точек и список квадратов, очищает текущую сцену и заново отображает переданные объекты. Точки отображаются в виде небольших окружностей, а квадраты - в виде прямоугольников с одинаковой шириной и высотой. Также в области визуализации реализованы масштабирование с помощью колеса мыши, перемещение сцены и отображение координат курсора. Это позволяет удобнее анализировать расположение объектов на плоскости. На текущем этапе визуализация используется для проверки корректности ввода точек. В дальнейшем она будет применяться для отображения решений, полученных генетическим алгоритмом.

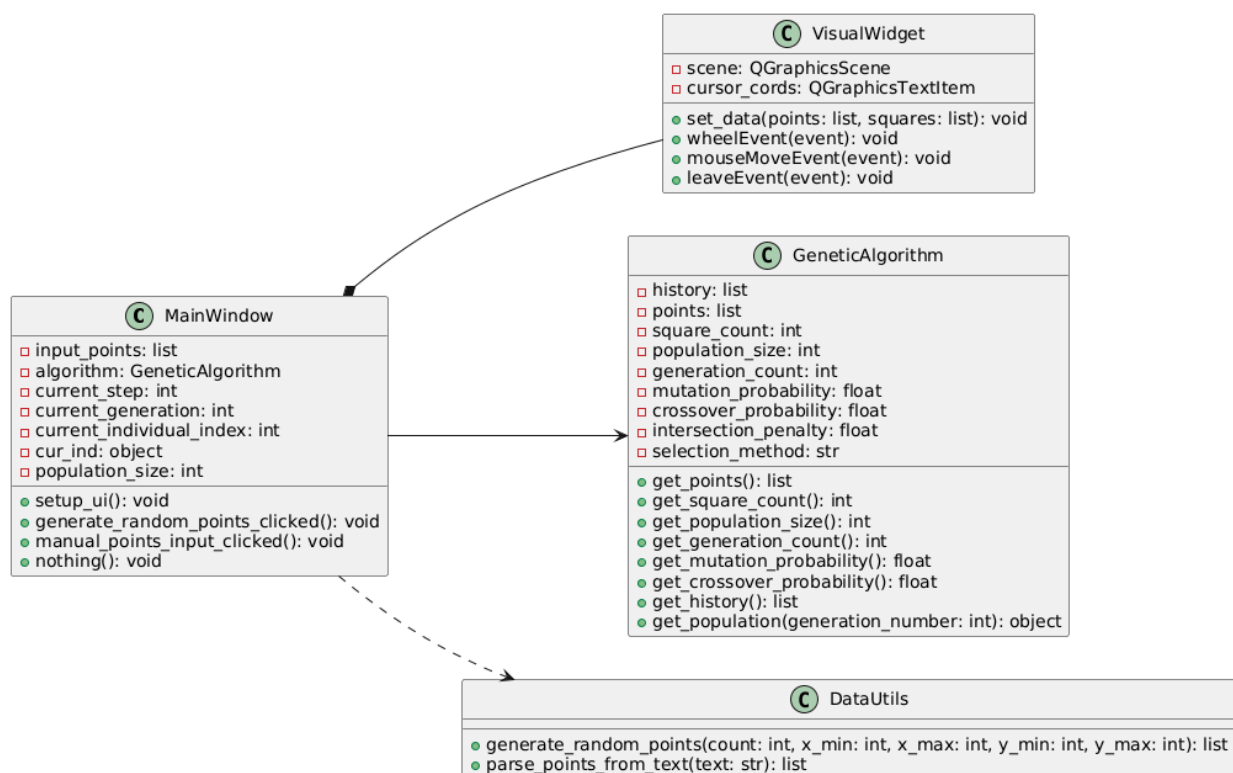


Рисунок 2 – UML-диаграмма текущей версии проекта

На рисунке 2 представлена UML-диаграмма текущей версии проекта. Диаграмма отражает структуру прототипа графического приложения и основные связи между классами.

Класс MainWindow отвечает за создание главного окна приложения, размещение элементов интерфейса и обработку действий пользователя. В нём хранятся входные точки, текущий выбранный шаг, индекс текущего индивида и ссылка на объект генетического алгоритма.

Класс VisualWidget отвечает за отображение точек и квадратов. Он содержит графическую сцену и методы для обновления изображения, масштабирования и отображения координат курсора.

Класс GeneticAlgorithm предназначен для хранения параметров генетического алгоритма и истории поколений. На текущем этапе он используется как основа для дальнейшей реализации алгоритма.

Класс DataUtils содержит вспомогательные методы для работы с входными данными.

2.5 План реализации генетического алгоритма.

Для решения задачи покрытия точек квадратами был выбран генетический алгоритм, так как задача является оптимизационной и требует подбора большого количества вещественных параметров. Входными данными алгоритма являются список точек на плоскости, количество квадратов, размер популяции, число поколений, вероятность мутации, вероятность скрещивания, штраф за пересечение квадратов и метод отбора родителей. Точки хранятся в виде пар координат (x, y) , без создания отдельного класса, поскольку точка в данной задаче не имеет собственного поведения и используется только в геометрических вычислениях.

Одна особь генетического алгоритма представляет собой один вариант расположения квадратов. Если пользователь задает M квадратов, то особь содержит набор $I = [s_1, s_2, \dots, s_M]$, где каждый квадрат задается тройкой параметров $s_j = (x_j, y_j, a_j)$. Здесь x_j и y_j - координаты левого нижнего угла квадрата, а a_j - длина его стороны. Таким образом, хромосома особи представляет собой набор вещественных значений, описывающих координаты и размеры всех квадратов.

Качество особи оценивается с помощью функции приспособленности $F(I) = C(I) - aK(I)$, где $C(I)$ - количество уникальных точек, покрытых квадратами данной особи, $K(I)$ - количество пар пересекающихся квадратов, а a - коэффициент штрафа за пересечение. Такая функция качества позволяет одновременно учитывать основную цель задачи - покрытие максимального количества точек - и ограничение на непересечение квадратов. Чем больше значение функции приспособленности, тем более удачным считается решение.

Работа генетического алгоритма планируется следующим образом: сначала создаётся начальная популяция случайных особей, затем для каждой особи вычисляется функция приспособленности, после чего выполняются отбор родителей, скрещивание, мутация и формирование нового поколения. В интерфейсе предусмотрен выбор метода отбора родителей. Основным

вариантом планируется турнирный отбор, так как он устойчиво работает даже при отрицательных значениях функции качества, которые могут возникать из-за штрафов. Также в архитектуре предусмотрены рулеточный и ранговый отбор. Для сохранения лучших решений будет использоваться элитизм, а история поколений будет храниться в виде списка снимков поколений внутри класса GeneticAlgorithm. Это позволит реализовать пошаговую визуализацию, построение графика изменения функции качества и возврат к предыдущим поколениям.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения учебной практики используются следующие технологии:

1. Python 3 - основной язык программирования проекта. Используется для реализации графического интерфейса, вспомогательных функций и генетического алгоритма.
2. PyQt5 - библиотека для создания графического интерфейса. Используется для построения главного окна, кнопок, полей ввода, таблицы, выпадающих списков и области визуализации.
3. Matplotlib - библиотека для построения графиков. Используется для отображения графика изменения функции качества по поколениям.
4. Git - система контроля версий. Используется для фиксации изменений, работы по веткам и отслеживания этапов разработки.
5. GitHub - платформа для хранения удалённого репозитория проекта и совместной работы над кодом.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв
о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(ая) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(ая) <результат практики>. Получаемую работу обучающийся(ая) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата> <подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А